WYDZIAŁ TELEKOMUNIKACJI, INFORMATYKI I ELEKTROTECHNIKI

Metody ochrony cyberprzestrzeni - projekt

Wykrywanie ataku DoS



Szymon Rykała

nr alb. 110757

Informatyka stosowana II st.

1. Czym jest atak DoS..... 2
2. Opracowany system..... 2
2.1. Opis..... 2
2.2. Architektura..... 3
2.3. Komponenty..... 3
2.3.1. Atakowany serwer..... 3
2.3.2. Kolektor NetFlow..... 4
2.3.3. DoS detektor..... 5
2.3.4. Wskaźnik ataku..... 5
3. Testy..... 7
3.1. Uruchomienie środowiska testowego..... 7
3.2. Wyniki..... 9
4. Wnioski..... 11

1. Czym jest atak DoS

Atak DoS (Denial of Service) to rodzaj cyberataku, który ma na celu uniemożliwienie dostępu do usługi, zasobu lub sieci poprzez przeciążenie ich i wywołanie zakłóceń w normalnym funkcjonowaniu. Ataki DoS mogą mieć różne formy i cele. Wszystkie jednak mają wspólny efekt polegający na utrudnieniu lub uniemożliwieniu dostępu do zasobów lub usług online. Istnieje wiele różnych rodzajów ataków DoS, w tym:

1. Flood: Atakujący generuje ogromny ruch sieciowy, który zalewa celową usługę lub sieć, co prowadzi do jej przeładowania i niemożności obsługi prawidłowych żądań.
2. ICMP Echo Request (ping flood): Atakujący wysyła duże ilości żądań ping do celu, co może prowadzić do przeładowania i spowolnienia sieci.
3. SYN Flood: Atakujący wysyła dużą ilość żądań SYN do serwera, co może prowadzić do wyczerpania zasobów serwera i utrudnienia dostępu do niego.
4. DNS Amplification: Atakujący wykorzystuje otwarte serwery DNS do generowania dużego ruchu na cel serwisu lub sieci.
5. HTTP Flood: Atakujący generuje duże ilości żądań HTTP w celu zasypywania serwera lub aplikacji webowej.

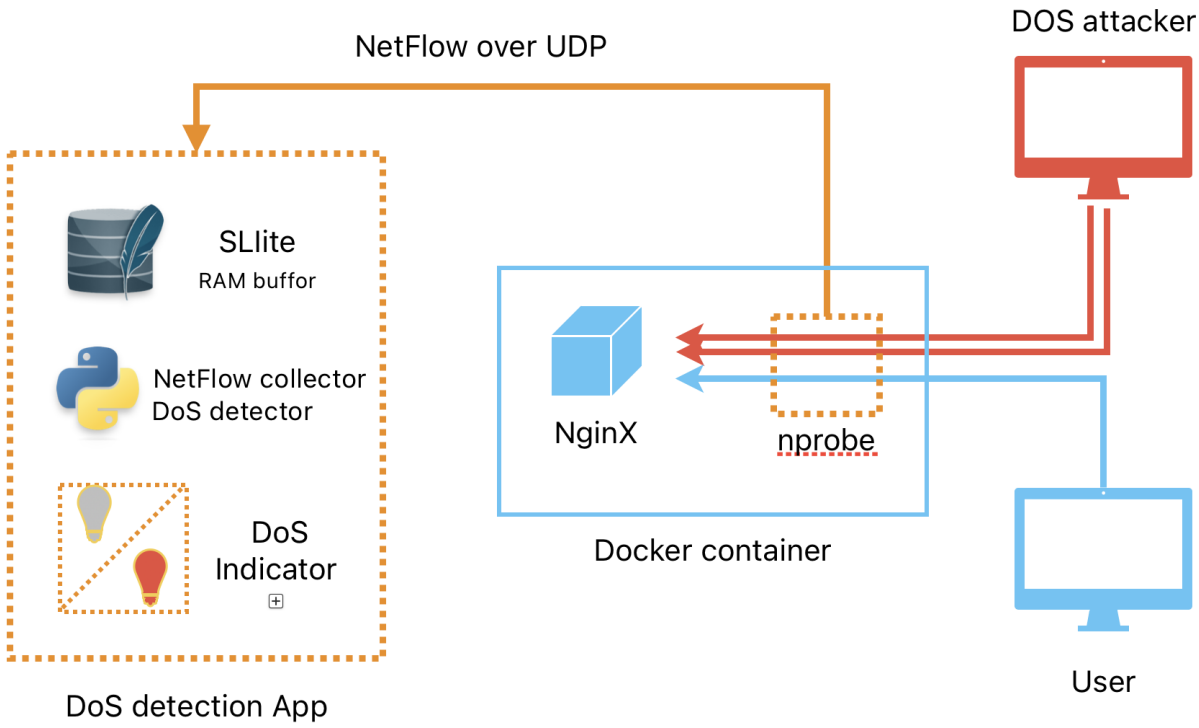
Ataki DoS mogą mieć różne cele, w tym wyłączenie konkurencyjnych usług online, niszczenie reputacji firmy, wymuszenie okupu lub sabotowanie działalności online. Aby się przed nimi bronić, organizacje często stosują strategie takie jak filtracja ruchu, wykrywanie anomalii i rozproszone systemy obrony (DDoS mitigation). Ataki DoS są nielegalne i podlegają ściganiu.
W przypadku niniejszej pracy, wykrywany będzie atak typu HTTP Flood.

2. Opracowany system

2.1. Opis

Przedstawiony system ma za zadanie w czasie niemal rzeczywistym wskazywać że nastąpił atak DoS. System ma działać z użyciem algorytmu opartego o z-score. Metryka ta wskazuje on jak bardzo konkretna wartość jest odchylona od średniej wartości danych. Wynik z-score może być umieszczony na krzywej rozkładu normalnego.
Gromadzony ruch pozyskiwany będzie w standardzie NetFlow. W przeciwieństwie do pcap (full packet capture) otrzymamy tylko metadane aktywności sieciowej a nie zawartość pakietów.

2.2. Architektura



Rys. 1. *Koncept architektury systemu wykrywania ataków DoS.*

Cały system jest oparty na wykorzystaniu danych zgromadzonych przez oprogramowanie nprobe firmy ntop, działającego zgodnie ze standardem NetFlow. Dane te zostaną zebrane na interfejsie sieciowym kontenera z uruchomionym wewnątrz, atakowanym serwerem.

Dane te przesyłane będą do aplikacji złożonej z trzech głównych komponentów:

1. Kolektora NetFlow z SQLite jako bufor w pamięci
2. Analizator ruchu.
3. Graficznej kontrolki alarmujące o wykryciu ataku.

2.3. Komponenty

2.3.1. Atakowany serwer

Atakowany serwer to Nginx z domyślną stroną startową. Umieszczony jest w kontenerze w formacie Docker. obiera on zapytania HTTP na porcie nr 80. Oprócz tego w kontenerze uruchomiony jest również nprobe - oprogramowanie zbierające ruch sieciowy na wskazanym interfejsie sieciowym. W trybie “probe”, wysyła zebrane dane w pakietach NetFlow na wskazany adres sieciowy.

nprobe.conf

configs > nprobe.conf

```
1  -i=lo
2  -n=host.docker.internal:5005
```

Zdj. 1. *Konfiguracja narzędzia "nprobe"*

Kontener uruchamiany jest z użyciem Podman'a - otwarto źródłowego oprogramowania służącego do uruchamiania i budowania kontenerów. Definicja kontenera znajduje się w pliku Dockerfile.

Dockerfile

Dockerfile > ...

```
1   FROM python:3.11.6-bullseye
2
3   EXPOSE 80
4
5   # setup ntop repo
6   RUN sed -i -E 's/^(deb[ \t]+[^\t ]+[ \t]+[^\t ]+)/\1 contrib/' /etc/apt/sources.list
7   RUN sed -i -E 's/^(deb-src[ \t]+[^\t ]+[ \t]+[^\t ]+)/\1 contrib/' /etc/apt/sources.list
8
9   RUN wget https://packages.ntop.org/apt-stable/bullseye/all/apt-ntop-stable.deb
10  RUN apt-get update && apt-get install ./apt-ntop-stable.deb -y
11
12  # install packages
13  RUN apt-get update && apt-get install iputils-ping nginx nprobe -y
14  RUN pip install netflow
15
16  # inject confis
17  COPY ./configs/nprobe.conf etc/nprobe/
18  COPY ./configs/nginx.conf etc/nginx/
19  COPY ./configs/startup /
20  RUN chmod +x startup
21
22  CMD [ "./startup" ]
```

Zdj. 2. *Konfiguracja obrazu kontenera.*

Kontener przy starcie uruchamia plik wykonywalny "./startup" - jest to zestaw komend *sh* uruchamiających serwer nginx oraz nprobe.

2.3.2. Kolektor NetFlow

Komponent ten odpowiedzialny jest za odbieranie pakietów wysyłanych przez nprobe i zapisywanie ich do bufora na dane w pamięci RAM zbudowanego na bazie SQLite. Uruchamiany jest on za pomocą kontrolki graficznej która zostanie omówiona w późniejszym rozdziale.

Kolektor uruchamiany jest na oddzielnym wątku. Możliwe jest zatrzymanie i wznowienie pracy w dowolnym momencie przy użyciu interfejsu graficznego.

2.3.3. DoS detektor

Detector podobnie jak kolektor uruchamiany jest na kolejnym wątku pobocznym. Co określony czas dokonuje on analizy wskazanej ilości próbek uzyskanych z bufora z danymi NetFlow - SQLite. Parametry te można regulować wedle uznania.
Dokonywana analiza polega na obliczaniu średniej okna pomiarowego oraz średniej z z-score kilku (4 oraz 6) ostatnich próbek. Jest to odchył od średniej wartości danych. Jeżeli wynosi on więcej niż określony próg, wyzwalany jest alarm - atak zostaje wykryty. Sygnalizowane jest to przez interfejs użytkownika.

```
def run_analysys(self):
    recent_stats = self.__store.run(GetStatsSince(2))
    if recent_stats:
        self.__store.run(DeleteAfter(CLEANING_TIME))
        self.traffic_data = (self.traffic_data + recent_stats)[-ANALYZER_MEMORY:]
        self.z_score = self.get_z_score(self.traffic_data)

        if self.traffic_data:
            few = mean(self.z_score[-4:])
            more = mean(self.z_score[-6:])

            if few > Z_SCORE_TH:
                self.__last = True
            elif more < few:
                self.__last = False

    return self.__last
```

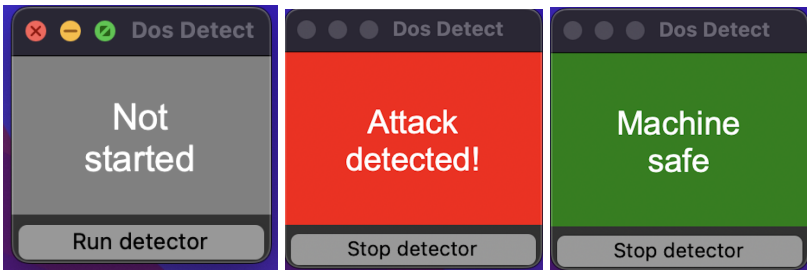
Zdj.3. *Metoda odpowiedzialna za analizę statystyk ruchu.*

Metoda przedstawiona na zdj. nr 3 odpowiedzialna jest za pobieranie statystyk ruchu z bufora danych NetFlow, wyliczanie z-score dla okna o zadanej długości oraz zdecydowanie na podstawie średniego odchyłu najnowszych wartości czy nastąpił atak, czy taki właśnie się zakończył. Jeśli zmienna prywatna "*__last*" przybiera wartość "*True*", oznacza to że wykryto atak.

2.3.4. Wskaźnik ataku

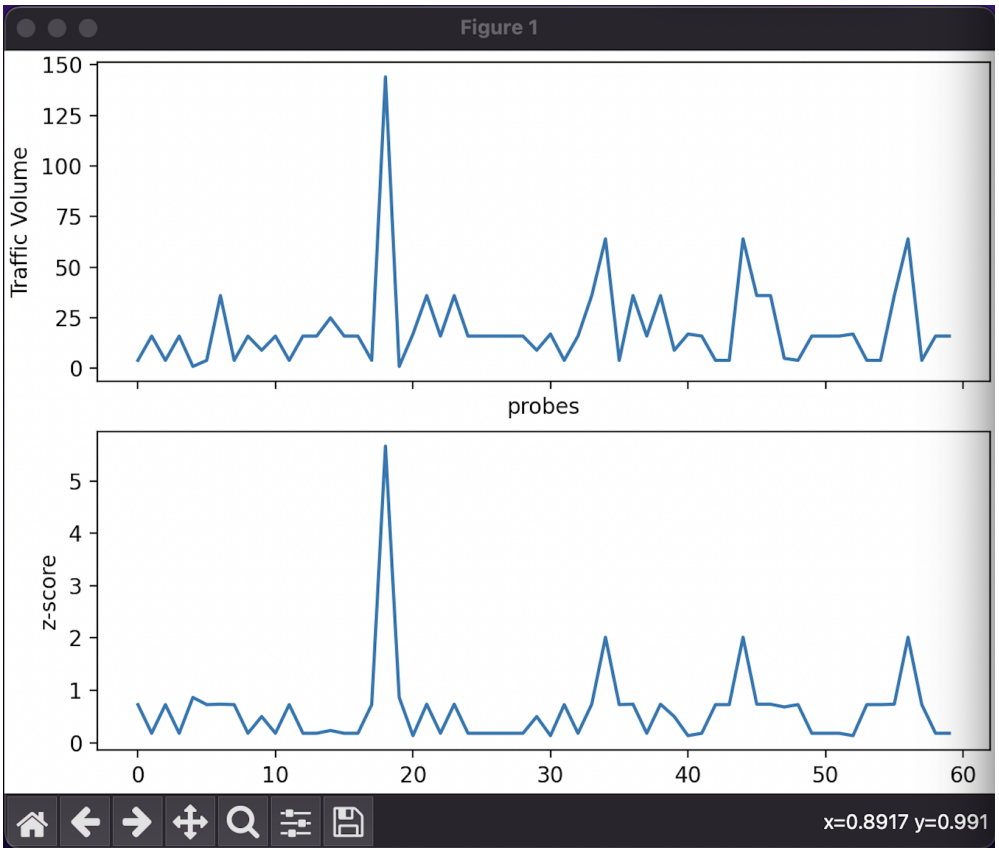
Kontrolka detektora ataku DoS ma postać graficzną. Posiada ona trzy stany:

1. Wykrywanie ataków wyłączone - sygnalizowane szarym kolorem kontrolki oraz napisem “Not started”.
2. Wykrywanie ataków jest włączone - w tym stanie uruchomione są wątki kolektora oraz detektora. Kontrolka przybiera wówczas dwa kolory:
    a. czerwony z napisem “Attack detected” - wykryto atak na obserwowane urządzenie,
    b. zielony z napisem “Machine safe” - brak ataków.



Zdj.4. *Stany wskaźnika ataku*

Oprócz powyższego wskaźnika, system oferuje również wykres ruchu sieciowego, oraz wykres z-score. Pokazuje on niemalże na żywo, natężenie ruchu na obserwowanym urządzeniu.



Zdj.5. *Wykres natężenia ruchu na urządzeniu.*

3. Testy

3.1. Uruchomienie środowiska testowego

Aby w pełni zasymulować otoczenie aplikacji webowej, konieczne było napisanie skryptu symulującego ruch sieciowy zwykłych użytkowników aplikacji oraz narzędzie do przeprowadzania szybkich ataków DoS.

Symulator zwykłego ruchu HTTP, co losową ilość milisekund, wysyła zapytanie do serwera nginX, czeka na odpowiedź, a następnie wysyła kolejne. Skrypt służący do ataku wykorzystuje niskopoziomowy interfejs sieciowy, aby wysłać zapytanie HTTP, lecz bez oczekiwania na odpowiedź serwera. Dzięki takiemu zabiegowi oraz wielowątkowości, jesteśmy w stanie wysyłać tysiące zapytań w ciągu sekundy.

```
21  def send_batch_request(per_thread):
22      for _ in range(per_thread):
23          try:
24              s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
25              s.connect((host, int(port)))
26              s.sendall(bytes(f"GET / HTTP/1.1\r\nHost: {host}\r\n\r\n", "utf-8"))
27              s.close()
28          except Exception as e:
29              print(e)
30          sleep(0.01)
31
32
33  threads = tuple(
34      Thread(target=lambda: send_batch_request(per_thread)) for _ in range(threads_count)
35  )
36  tuple(th.start() for th in threads)
37  tuple(th.join() for th in threads)
```

Zdj.6. *Sposób wykonywania zapytań w przypadku ataku.*

W celu szybkiego uruchamiania środowiska testowego, zdecydowano się na napisanie skryptu z komendami, które stanowią abstrakcję pomiędzy poszczególnymi komponentami a testerem.

Dostępne komendy to:

- *./run build <nazwa tagu obrazu>* - buduje obraz kontenera z podanym tagiem
- *./run container <nazwa tagu obrazu>* - uruchamia kontener z użyciem obrazu z identycznym tagiem
- *./run detector* - uruchamia aplikację wykrywającą ataki DoS
- *./run client-sim* - uruchamia symulator klienta HTTP
- *./run attack [--count <int>, --threads <int>]* - uruchamia atak HTTP flood o podanej ilości requestów rozłożonych na zdefiniowaną ilość wątków.

$ run

```
$ run
# !/bin/bash

error_message="One of options is required: [detector, build, container, client-sim, attack]"
PORT=3000
NETFLOW_PORT=5005
IP=localhost

case "$1" in
build)
    podman image rm dos-attack-target-img:latest
    podman build --tag dos-target-img:$2 .
    ;;

container)
    podman run \
        -it \
        --rm \
        -p $PORT:80 \
        --cap-add NET_RAW \
        --cap-add CAP_NET_RAW \
        --name dos-target-container \
        dos-target-img:$2
    ;;

detector)
    export COLLECTOR_IP=$IP
    export COLLECTOR_PORT=$NETFLOW_PORT
    python3 ./app
    ;;

client-sim)
    python3 traffic/client.py \
        --host $IP \
        --port $PORT \
        --stats ./data/user_traffix_stats.csv
    ;;

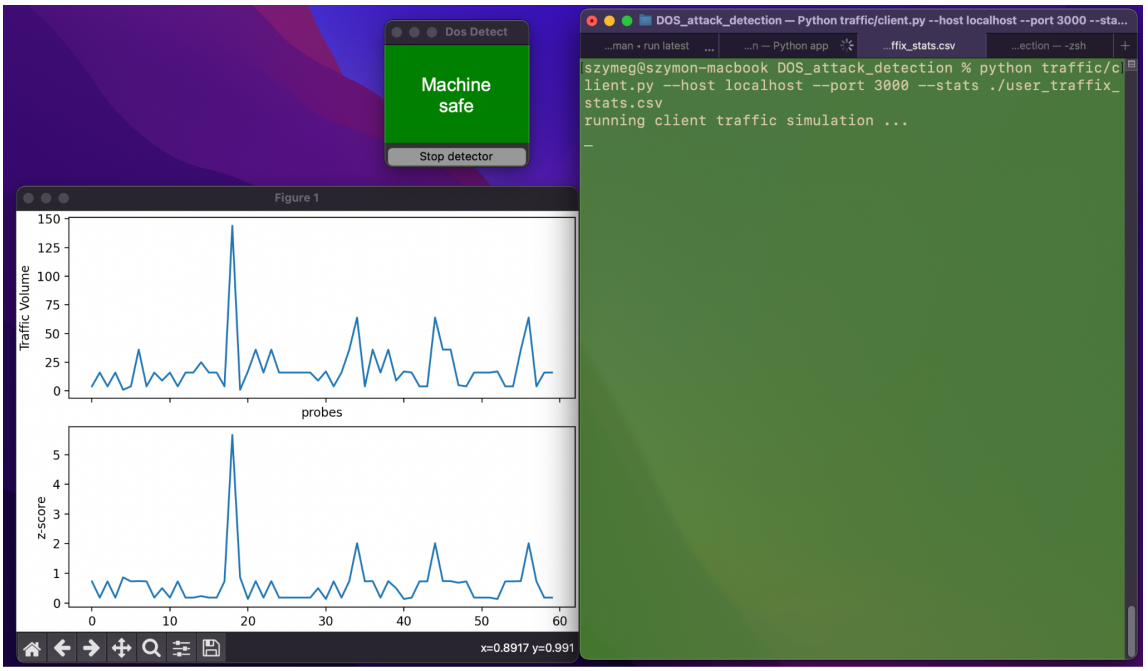
attack)
    python3 traffic/attack.py \
        --host $IP \
        --port $PORT \
        $2 $3 \
        $4 $5
    ;;

*)
    echo $error_message
    exit 1
    ;;
esac
```

Zdj. 7. *Skrypt z komendami uruchomieniowymi.*

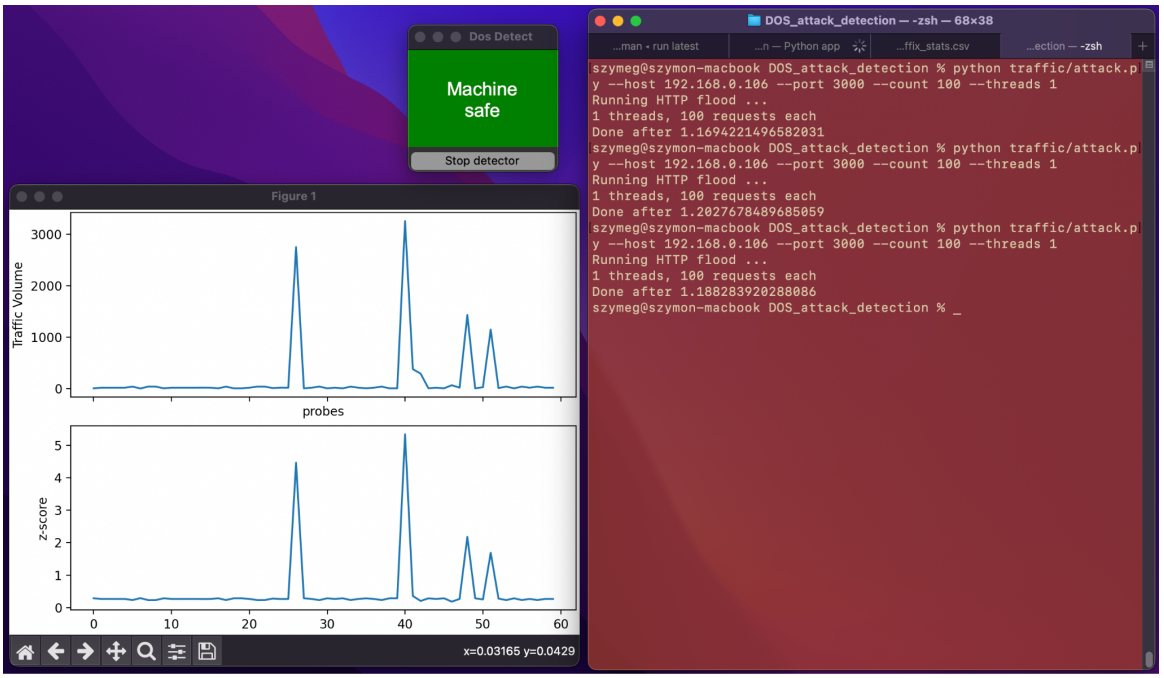
3.2. Wyniki

Podczas symulowanego ruchu użytkowników, detektor wskazuje że maszyna jest bezpieczna co jest jak najbardziej poprawnym rezultatem jego działania.



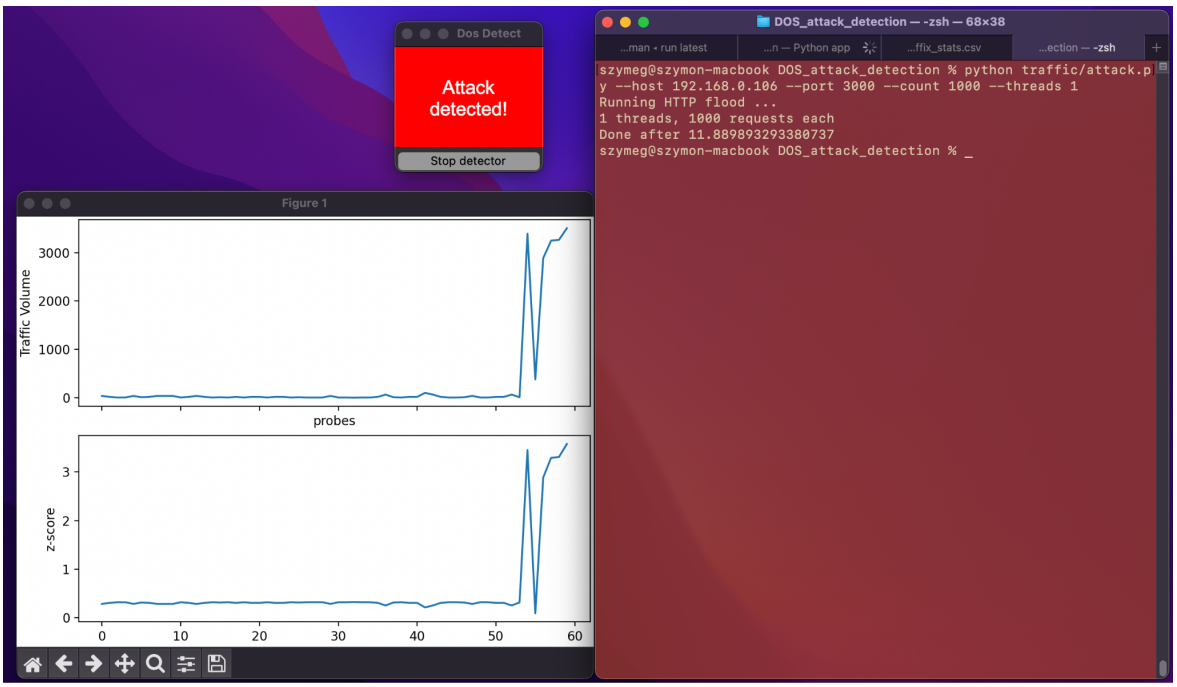
Zdj. 8. *Detektor włączony przy symulowanym ruchu użytkowników.*

Podczas działanie aplikacji sieciowych, może zdażyć się tak, że w pewnych sytuacjach dochodzić będzie do nagłych, krótkich skoków w natężeniu ruchu. Podczas takiego zjawiska, pożądanym zachowaniem systemu wykrywającego ataki DoS jest nie podnoszenie alarmu.



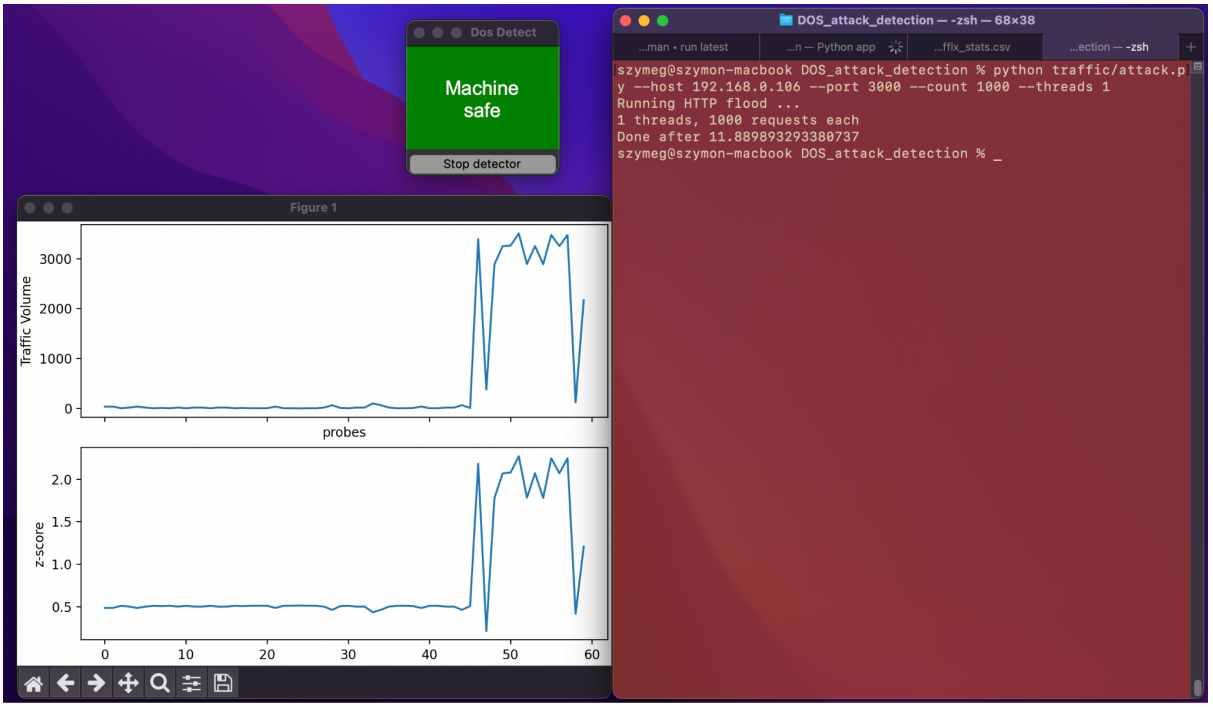
Zdj. 9. *Detektor w przypadku pojedynczych pikach natężeniach ruchu.*

Kiedy zostanie przeprowadzony atak, system powinien ostrzec o tym użytkownika w odpowiedni sposób.



Zdj. 10. *Detektor podczas wykrytego ataku.*

Po wykryciu ataku, i odpowiednim jego zasygnalizowaniu, system wychodzi ze stanu alarmu jeśli tylko wykryje że się on zakończył - jeżeli natężenie ruchu spadnie.



Zdj. 11. *Detektor po wykryciu zakończenia ataku.*

4. Wnioski

Detektor spełnia swoje zadanie i wykrywa przeprowadzane ataki DoS. Należy jednak pamiętać że jest to projekt mający przedstawić jedną z metod przewidywania tego typu ataków.
Wykorzystana metryka z-score jest dobrym narzędziem do wykrywania odchyleń w natężeniu ruchu. Po ustawieniu odpowiedniego progu tej metryki, możemy wskazać kiedy następuje atak. W komercyjnych rozwiązaniach nie może to być jednak jedyny wskaźnik. Metrykę tę można w prosty sposób unieszkodliwić. Aby to zrobić należy w sztuczny sposób podnosić średnią zapytań w taki sposób aby nie wystąpił nagły pik w natężeniu ruchu. Metryka z-score jest wówczas bezużyteczna. Komercyjny system powinien brać pod uwagę wiele parametrów ruchu sieciowego takiego jak czas odpowiedzi, wielkość pakietów etc.